Sistema de ingesta continua con Kafka y backups diarios con Salt Minion

Raúl Cristian Dumitru Tanase
2º CFGM SMR
01/03/2026

ÍNDICE

1. Cuestiones a resolver **2**
- ¿Qué es Apache Kafka y para qué tipo de sistemas se utiliza? 2
- ¿Quién desarrolló Apache Kafka originalmente y en qué empresa nació el proyecto? 2
- ¿Qué significa que Kafka sea una plataforma de streaming distribuido? 2
- ¿Qué es un topic en Kafka y para qué sirve? 2
- ¿Qué es un producer en Kafka y cuál es su función? 2
- ¿Qué es un consumer y cómo recibe los mensajes? 3
- ¿Qué es una partition y por qué mejora el rendimiento? 3
- ¿Qué papel cumple un broker dentro de Kafka? 3
- ¿Qué es Zookeeper y por qué se ha usado tradicionalmente con Kafka? 3
- ¿En qué tipo de aplicaciones reales se utiliza Kafka (ejemplos)? 3
- ¿Si tengo que utilizar otra base de datos asociada a Kafka, para que sirve entonces? 3
- ¿Qué es un Minion en el contexto de SaltStack? 3
- ¿Qué relación existe entre un Minion y un servidor Master? 3
- ¿En qué tipo de equipos se puede instalar un Minion (PC, servidor, sistema operativo)? 3
- ¿Para qué tareas se utiliza un Minion en la administración de sistemas? 3
- ¿Cómo se comunica un Minion con el Master? 4
- ¿Puede un Minion ejecutar copias de seguridad? Explica cómo. 4
- ¿Qué ventajas tiene usar Minions frente a administrar equipos uno a uno manualmente? 4
- ¿Qué tipo de acciones se pueden automatizar mediante un Minion? 4
- ¿Qué ocurre si el Minion pierde la conexión con el Master? 4
- ¿En qué entornos reales (empresa, centro educativo, servidores) sería útil usar Minions? 4

2. Desarrollo del proyecto **5**
1. Introducción 5
2. Flujo de trabajo 5
3. Infraestructura 6
4. Infraestructura a nivel técnico 7
5. Creación de la infraestructura 8

1. Cuestiones a resolver

- ***¿Qué es Apache Kafka y para qué tipo de sistemas se utiliza?***

    - Apache Kafka es un servicio de streaming de eventos que mueve grandes volúmenes de datos de forma rápida.

- ***¿Quién desarrolló Apache Kafka originalmente y en qué empresa nació el proyecto?***

    - El desarrollador original fue Jay Kreps con Neha Narkhede y Jun Rao. Lo importante realmente es que nació en LinkedIn por una necesidad urgente de mover grandes volúmenes entre sistemas.

- ***¿Qué significa que Kafka sea una plataforma de streaming distribuido?***

    - Streaming viene porque mueve datos en tiempo real y distribuido porque, para ser más eficiente no usa solamente una máquina sino que lo distribuye en Brokers y esos Brokers en partitions.

- ***¿Qué es un topic en Kafka y para qué sirve?***

    - Un topic es el canal mediante el cual se mueven los batch de datos. Se pueden crear indefinidos para separar los datos.

- ***¿Qué es un producer en Kafka y cuál es su función?***

    - El producer es básicamente un sistema que envía datos, elige el topic correcto, el broker más eficiente y envía los datos a Kafka.

- ***¿Qué es un consumer y cómo recibe los mensajes?***

    - El consumer es quien lee los eventos del producer y marca un offset donde termina su lectura de datos dado que los datos no se borran del topic una vez enviados.

- ***¿Qué es una partition y por qué mejora el rendimiento?***

    - Una partition no deja de ser una mejora en la eficiencia de un topic al dividirlo en varias vías que pueden ser asignadas a diferentes brokers.

- ***¿Qué papel cumple un broker dentro de Kafka?***

    - El Broker es quien recibe, almacena y le entrega los mensajes al consumer. Es el servidor intermediario.

- ***¿Qué es Zookeeper y por qué se ha usado tradicionalmente con Kafka?***

    - Es un servicio de coordinación desarrollado por Apache también. Básicamente decide qué Broker prevalece como el principal en un topic.

- ***¿En qué tipo de aplicaciones reales se utiliza Kafka (ejemplos)?***

    - Como este mismo proyecto, un pipeline de datos entre microservicios.

- ***¿Si tengo que utilizar otra base de datos asociada a Kafka, para qué sirve entonces?***

    - Porque sin Kafka no puedo mover mis datos entre sistemas. Supongamos que mi arduino me da unos datos. Kafka es verdad que los almacena pero su objetivo es ese, no tiene una zona centralizada para ver esos datos dado que no es una BD. Lo que hace es transportar el mensaje a la propia BD para que lo inyecte en una tabla.

- ***¿Qué es un Minion en el contexto de SaltStack?***

    - Es un servicio bajo el control de Saltstack. Recibe las órdenes del máster (salt) y las ejecuta.

- ***¿Qué relación existe entre un Minion y un servidor Master?***

    - El máster centraliza todos los minion y da las órdenes indicadas a cada uno, el minion simplemente ejecuta.

- ***¿En qué tipo de equipos se puede instalar un Minion (PC, servidor, sistema operativo)?***

    - Básicamente en todos los OS que existen y en todos los equipos informáticos menos obviamente router, switches y dispositivos de esa familia.

- ***¿Para qué tareas se utiliza un Minion en la administración de sistemas?***

    - Puede gestionar paquetes, gestionar servicios etc…

- ***¿Cómo se comunica un Minion con el Master?***

    - Con una librería de red de muy baja latencia llamada ZeroMQ.

- ***¿Puede un Minion ejecutar copias de seguridad? Explica cómo.***

    - Sí, básicamente el máster tiene un script que se debe ejecutar cada x segundos y se lo hace ejecutar al minion al que nosotros deseemos asignarle esa acción

- ***¿Qué ventajas tiene usar Minions frente a administrar equipos uno a uno manualmente?***

    - Que los minions se centralizan en el máster de SaltStack y tenemos acceso desde solo un sitio.

- ***¿Qué tipo de acciones se pueden automatizar mediante un Minion?***

    - Literalmente las que sean, es cuestión de darle una tarea. No tiene un límite perse, simplemente hace caso.

- ***¿Qué ocurre si el Minion pierde la conexión con el Master?***

    - Deja de recibir órdenes del máster y por ende se para.

- ***¿En qué entornos reales (empresa, centro educativo, servidores) sería útil usar Minions?***

    - La pregunta es al revés, en cuales no sería útil. Pero sobre todo cuando hay muchos servicios distintos o servidores y merece la pena centralizarlos.

2. Desarrollo del proyecto

1. Introducción

Kafka es un sistema de mensajería distribuida para mover grandes cantidades de datos entre sistemas de forma veloz y eficiente. A groso modo tiene tres categorías principales; Mensajería (Comunicación entre microservicios), Streaming (Procesar eventos en tiempo real) e Integración en sistemas (Conectar bases de datos entre otros).

Algunos ejemplos reales de uso pueden ser; Recoger logs de multiples servidores simultáneamente, telemetría de dispositivos IOT (Nuestro caso) o eventos en apps.

SaltStack es una herramienta de automatización que permite centralizar multiples servidores en un mismo punto central. Es una arquitectura Master-Minion donde el master es el servidor central que da órdenes y el Minion son los servidores anexos que reciben y ejecutan esas mismas órdenes.

En este caso y a groso modo también lo podemos dividir en tres categorías principales: Automatización (Instalación de software, cambiar configuraciones de servidores sin acceder a ellos u otros), Administración centralizada (Observar todo el estado de una infraestructura desde un solo sitio) y Ejecución de tareas (Copias de seguridad, parches de seguridad etc).

Algunos ejemplos reales pueden ser; Desplegar una nueva versión de una app con un comando o automatizar backups nocturnos (Nuestro caso).

2. Flujo de trabajo

Para realizar este proyecto primero hay que entender el flujo de trabajo que conlleva la unión de todos estos servicios.

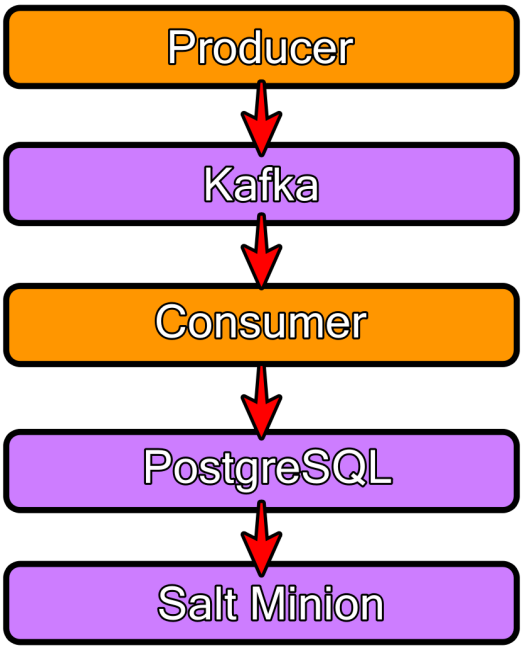
1. ¿Cual es el objetivo?

El objetivo de este proyecto es simular entradas de datos físicos obtenidos atraves de sensores arduinos con scripts de python que posteriormente se almacenen en una base de datos para ser accesibles en cualquier momento. Además, con un sistema de copias de seguridad diario para evitar perder información crucial del proyecto.

2. Esquema del flujo:

Tal como se puede observar en este esquema en primer lugar tenemos un script de python que actua como “producer”. Es decir, quien proporciona los datos. Por otra parte tenemos al propio kafka que hace de interlocutor entre el producer y el “consumer”. El consumer es otro

script de python que todo lo que recibe de kafka lo ejecuta en la base de datos. En este caso al ser una línea json que se inyecta en la base de datos automaticamente se ordena en la misma. El siguiente elemento que interviene es el container de postgreSQL que almacena los datos. Por último, tenemos a salt minion que realiza las copias de seguridad diariamente o manualmente como veremos posteriormente.



3. Infraestructura

Para explicar la infraestructura del proyecto se va a explicar carpeta a carpeta y cómo se relacionan entre ellas.

1. /SOR/compose

El primer archivo que encontramos aquí es el “compose.yaml”. Este archivo nos permite configurar diferentes contenedores con microservicios en un mismo grupo para ejecutarlo a la vez y que todos los servicios se levanten al mismo tiempo. Este archivo nos permite centralizar la estructura y configurar la relación entre servicios. En el **anexo 1** se puede observar el código y su explicación paso a paso.

El segundo archivo es “.env” que contiene configuración que necesita ser más accesible, como por ejemplo, las credenciales de la base de datos o el nombre del topic que vamos a usar en kafka. Es información que quiza tengamos que cambiar y por ello se crea un archivo aparte para dicha información.

2. /SOR/db

El único archivo que encontramos aquí es “init.sql” que contiene la configuración de la base de datos. Esto nos permite recrear el contenedor cuantas veces queramos sin tener que preocuparnos de crear la tabla de datos nuevamente ya que este documento la creará automáticamente al levantar el servicio.

3. /SOR/scripts

El primer archivo de este directorio es “consumer.py”. Este script lo pongo el primero dado que es el que debe ser ejecutado antes de ningún otro. Esto se debe a que es un script de python que lee las comunicaciones de Kafka y las inyecta en la base de datos. Si ejecutamos el producer antes del consumer se perderán datos.

El segundo archivo del directorio es “producer.py” que simula la producción de datos de un supuesto arduino. Este script genera cada 2 segundos una línea de código json con datos como; Fecha, temperatura y humedad.

El tercer archivo es “requierements.txt”. Es un documento de texto que ayuda a crear un entorno venv de python con las librerías necesarias para ejecutar ambos scripts.

Por último tenemos el subdirectorio venv que es donde se almacenan dichas librerías mencionadas anteriormente para ejecutar los scripts.

4. /SOR/salt

En primer lugar tenemos el subdirectorio “minion.d” que contiene la configuración del minion en la cual se establece la automatización de las copias de seguridad.

En segundo lugar tenemos el archivo “backup_db.sh” que aguarda la configuración de el cómo se realiza dicha copia de seguridad. Extensión, nombre etc.

Por último tenemos el dockefile que establece una imagen de ubuntu preconfigurada con lo necesario para ejecutar el Salt Master y los Minion.

4. Infraestructura a nivel técnico

En este apartado vamos a recopilar uno a uno los servicios que hemos configurado gracias a los archivos explicados anteriormente. Una explicación más enfocada hacia la herramienta en sí.

1. Docker Desktop

Docker es una herramienta polivalente que posee infinidad de aplicaciones. En este caso nos ha sido válida para desplegar contenedores de GNU/Linux donde almacenar dichos microservicios sin acceder directamente a la máquina sino guiandonos de forma externa mediante comandos en una terminal. Estos containers los hemos agrupado en un compose definido en el apartado de infraestructura y en el **anexo 1.**

2. Kafka y Zookeeper

Como hemos mencionado también anteriormente, Kafka se está usando en este proyecto como interlocutor entre los datos presuntamente reales (que son simulados por un script) y la base de datos.
Zookeeper es el máster del cual depende Kafka y se encarga de coordinar la comunicación eligiendo quién es el Broker que lidera la intervención.

Entendemos un Broker como un nodo que usa Kafka para realizar la comunicación. Recibe los mensajes, los organiza en topics y particiones y posteriormente los sirve al consumer.

Por otro lado, los topics podemos entenderlos como el canal mediante el cual realizamos la comunicación.

Por último ese mismo topic se divide en partitions que permiten un mejor rendimiento y mayor escalabilidad.

3. PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos, sistema que nos permite crear y relacionar tablas entre sí. En este caso solo hemos precisado de una tabla dado que hemos introducido todos los posibles valores agrupados en una tabla. A pesar de ello, en un proyecto real lo más eficiente sería una correcta distribución de los datos.

Lo más importante de PostgreSQL es que admite tipos de datos avanzados como diccionarios JSON que interpretados correctamente se adhieren a la tabla generando los datos generados por el producer e ingestados por el consumer posteriormente.

4. Saltstack

Anteriormente hemos podido hacer un poco de introspección en las funciones que cumple saltstack y la definición de un minion. Saltstack actúa en nuestro caso como el agente que centraliza los minion que precisemos en nuestra infraestructura.

En este caso, solamente hemos necesitado un solo minion encargado de realizar las copias de seguridad. Un proyecto real y eficiente distribuye las tareas en diversos minion.

5. Creación de la infraestructura

1. Docker desktop

Con el compose ya creado vamos a acceder a nuestro terminal preferido para levantar el grupo de microservicios con el comando:

```
docker compose up -d
```

Si algún contenedor fallará durante la inicialización podemos crearlo para que vuelva a cargar. Lo haremos con el siguiente comando:

```
docker compose up -d --force-recreate {container}
```

En caso de necesitar parar el compose (no tendría porque). Usaremos este comando **sin parametros extra** ya que podrían borrarse todos los volúmenes creados por el compose que almacenan información de configuración o la propia base de datos:

```
docker compose down
```

Si quisiéramos ver los servicios activos del stack, usaremos el comando:

```
docker compose ps
```

Por último, en caso de querer ver logs de Docker en tiempo real, usaremos el comando:

```
docker logs -f {container}
```

2. Kafka

Una de las cosas que hemos dejado configuradas previamente en nuestro archivo /compose/.env es el nombre que usará nuestro topic de Kafka. Esto no quiere decir que el topic se crea automáticamente sino que estamos predefiniendo ese nombre para el propio topic.

Por ende, para crear el topic no entraremos directamente al container sino que podemos realizarlo desde el exterior de la misma con el comando:

```
docker exec -it SOR-kafka kafka-topics --bootstrap-server localhost:9092
--create --topic SOR_sensores --partitions 1 --replication-factor 1
```

Donde:

- docker exec -it: Es el acceso desde el exterior al container.
- SOR-kafka: Es el contenedor.
- kafka-topics: Herramienta para gestionar topics.
- –bootstrap-server localhost:9092: Dirección IP del broker.
- –create –topic SOR_sensores: Creación del propio topic con el nombre del .env.
- –partitions: Definición del número de partitions.
- –replication-factor: Número de réplicas de cada partición.

Si quisiéramos comprobar la correcta creación del topic lo haremos con el comando:

```
docker exec -it SOR-kafka kafka-topics --bootstrap-server localhost:9092
--list
```

3. Scripts con python

En este caso primero vamos a ver cómo instalar las dependencias necesarias. Usaremos el comando:

```
pip install -r requirements.txt
```

Para ejecutar el consumer (es importante ejecutarlo primero dado que sino se pueden perder datos por el camino):

```
py consumer.py
```

Y para activar el producer:

```
py producer.py
```

4. PostgreSQL

Para acceder a la base de datos lo haremos mediante su gestor adminer, alojado en el puerto 8081.

```
localhost:8081
```

En el menú de Adminer usaremos las credenciales del archivo /compose/.env.

Para ver los ultimos registros usaremos el comando el apartado de SQL command de Adminer:

```
SELECT * FROM measurements ORDER BY ts DESC LIMIT {cantidad de logs};
```

5. Saltstack

Los archivos de configuración del directorio ya crean todo sin necesidad de ejecutar nada en el container (a diferencia de kafka que si lo precisa). Sin embargo podemos usar los siguientes comandos de utilidad.

Comando para visualizar la configuración de la creación automática de backups:

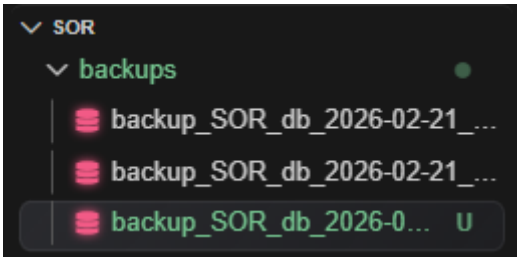
```
docker exec -it SOR-salt-minion salt-call --local config.get schedule
```

O por otra parte, si fuera necesario, ejecutar una copia de seguridad manual:

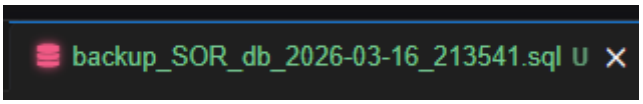
```
docker exec -it SOR-salt-minion salt-call --local cmd.run "bash
/srv/salt/backup_db.sh"
```

Como explicación breve del comando podemos entender que ejecutamos un comando desde el exterior del container para que el minion se autogestione (salt-call, si fuera el máster sería salt solamente) usando un comando exclusivamente en un entorno bash para ejecutar la ruta que aparece en el comando que genera dicho backup.

Como podemos observar, esta operación genera los backups perfectamente




```
PS C:\SOR\compose> docker exec -it SOR-salt-minion salt-call --local cmd.run "bash /srv/salt/backup_db.sh"
local:
    [INFO] Creating backup: /backups/backup_SOR_db_2026-03-16_213541.sql
    [OK] Backup created: /backups/backup_SOR_db_2026-03-16_213541.sql
    -rw-r--r-- 1 root root 796K Mar 16 21:35 /backups/backup_SOR_db_2026-03-16_213541.sql
PS C:\SOR\compose>
```



Las imagenes del output de cada comando están en una "galería" del [README.md](#) del repositorio de github.

6. *Conclusión personal*

Personalmente he quedado sorprendido con la facilidad que otorga tanto Kafka como Salt para usar su microservicio, con una breve configuración podemos mover datos sin problemas entre un diccionario y una base de datos. Además, Docker lo siento desde este proyecto como un must en cualquier tipo de despliegue de microservicios dada la claridad que proyecta sobre infraestructuras que sin el uso de la herramienta parecen bastante más caóticas.

Por otra parte, decir que no he usado la IA sería mentir descaradamente dado que hoy en día la sentimos una parte esencial de nuestra vida. En esta etapa donde han habido bastantes examenes y trabajos por hacer, aprender, por ejemplo, python al nivel necesario para hacer los scripts que he usado se hace imposible dado que no existe tiempo físico para ello. El problema viene realmente cuando uno hace control c + control v sin mirar realmente lo que está haciendo y sin entenderlo como para defenderlo delante de cualquier persona. Algo que me caracteriza en este aspecto, es que aunque uso la IA, no dejo que me domine sino que intento aprovecharla al máximo y sumar a mis conocimientos. Quizá no sé programar los scripts que he subido al repositorio pero aseguro saber leerlos y haberme informado de que hace cada parte. Porque se trata de eso, de aprender lo máximo posible con cada tarea que se hace aunque involucre a la IA. Es una herramienta muy poderosa, quien es inepto y la usa, será más inepto aún. Sin embargo, quien es bueno y la usa, se volverá aún más bueno.

Por último, opinó que integrar la IA en las correcciones lejos de ser beneficioso es necesario. Hoy en día gracias a Claude se pueden hacer cosas que no imaginabamos con la IA hace meses. Usar Claude entrenandolo con los parametros de cada profesor para que corrija de la manera que se solicita ahorraría mucho tiempo que muchas veces los profesores no tienen. Esto es como la pregunta anterior, está bien usar la IA, bien usada. Meter un examen a ChatGPT y esperar que lo haga bien es como tirar una moneda al océano atlantico y esperar encontrarla.

7. *Webgrafía*

1. Claude:
- Ayuda para generar los scripts de python y sobre todo para que me explique el por qué de cada línea.

2. ChatGPT
- Me ha ayudado a aprender qué es cada microservicio y para qué son usados realmente en el entorno profesional. Además, me ha ayudado a organizarme y a llevar un buen ritmo de trabajo.

3. Configuración de los topics mediante documentación oficial

4. Orientación hacia los compose de Docker